

Arrays and Vectors

Victor Eijkhout, Susan Lindsey

Fall 2026

last formatted: September 22, 2026

1. Do not use these arrays:

```
1 float x[5];  
2 double *xx = malloc( stuff );
```

- New mechanisms are better.

2. Short vectors

Short vectors can be created by enumerating their elements:

Code:

```
1 // array/shortvector.cpp
2 #include <vector>
3 using std::vector;
4
5 int main() {
6     vector<int> evens{0,2,4,6,8};
7     vector<float> halves =
8         {0.5, 1.5, 2.5};
9     auto halffloats =
10         {0.5f, 1.5f, 2.5f};
11     println( "Elt zero from {} is {}",
12             evens, evens.at(0) );
13     return 0;
14 }
```

Output:

```
1 ./shortvector
2 Elt zero from [0, 2,
    ↪4, 6, 8] is 0
```

3. Range over elements

You may have learned:

```
1  for ( int i=0; i<x.size(); ++i ) {  
2      auto value = x[i];  
3      // something with value
```

Better:

A range-based for loop gives you directly the element values:

```
1  vector<float> my_data(N);  
2  /* set the elements somehow */;  
3  for ( float e : my_data )  
4      // statement about element e
```

Here there are no indices because you don't need them.

4. Range over elements, version 2

Same with auto instead of an explicit type for the elements:

```
1 for ( auto e : my_data )  
2    // same, with type deduced by compiler
```

5. Range over elements

Finding the maximum element

Code:

```
1 // array/dynamicmax.cpp
2 vector<int> numbers = {1,4,2,6,5};
3 int tmp_max = -2000000000;
4 for (auto v : numbers)
5     if (v>tmp_max)
6         tmp_max = v;
7 println( "Max over {} is {}",
8         numbers,tmp_max );
```

Output:

```
1 Max over [1, 4, 2,
    ↪6, 5] is 6
```

Exercise 1

Indicate for each of the following vector operations whether you prefer to use an indexed loop or a range-based loop. Give a short motivation.

- Count how many elements of a vector are zero.
- Find the location of the last zero.

6. Range over vector denotation

Code:

```
1 // array/rangedenote.cpp
2 for ( auto i : {2,3,5,7,9} )
3     print( "{}",i );
4 println();
```

Output:

```
1 2,3,5,7,9,
```

7. Vector definition

Definition and/or initialization:

```
1 #include <vector>
2 using std::vector;
3
4 vector<type> name;
5 vector<type> name(size);
6 vector<type> name(size, init_value);
```

where

- `vector` is a keyword,
- `type` (in angle brackets) is any elementary type or class name,
- `name` of the vector is up to you, and
- `size` is the (initial size of the vector). This is an integer, or more precisely, a `size_t` parameter.
- Initialize all elements to `init_value`.
- If no default given, zero is used for numeric types.

8. Accessing vector elements

Square bracket notation (zero-based):

Code:

```
1 // array/assign.cpp
2 vector<int> numbers = {1,4};
3 numbers[0] += 3;
4 numbers[1] = 8;
5 println
6   ( "{},{}",
7     numbers[0],numbers[1]);
```

Output:

1 4,8

9. Accessing vector elements

With bound checking:

Code:

```
1 // array/assign.cpp
2 vector<int> numbers = {1,4};
3 numbers.at(0) += 3;
4 numbers.at(1) = 8;
5 println
6   ( "{},{}",
7     numbers.at(0), numbers.at(1));
```

Output:

```
1 4,8
```

Safer, slower.

(Remember Knuth about optimization.)

10. Vector elements out of bounds

Square bracket notation:

Code:

```
1 // array/assignoutofboundbracket.cpp
2 vector<int> numbers = {1,4};
3 numbers[-1] += 3;
4 numbers[2] = 8;
5 println
6   ( "{},{}",
7     numbers[0],numbers[1]);
```

Output:

```
1 1,4
```

11. Vector elements out of bounds

With bound checking:

Code:

```
1 // array/assignoutofboundatfun.cpp
2 vector<int> numbers = {1,4};
3 numbers.at(-1) += 3;
4 numbers.at(2) = 8;
5 println
6   ( "{},{}",
7     numbers.at(0), numbers.at(1));
```

Output:

```
1 terminate called
   ↳ after throwing
   ↳ an instance of
   ↳ 'std::out_of_range'
2   what():
   ↳ vector::_M_range_check:
   ↳ __n (which is
   ↳ 18446744073709551615)
   ↳ >=
   ↳ this->size()
   ↳ (which is 2)
3 make[1]: ***
   ↳ [Makefile:109:
   ↳ run_assignoutofboundatfu
   ↳ Aborted (core
   ↳ dumped)
```

12. Range over elements by reference

Range-based loop indexing makes a copy of the vector element. If you want to alter the vector, use a reference:

```
1 for ( auto &e : my_vector )
2   e = ....
```

Code:

```
1 // array/vectorrangeref.cpp
2 vector<float> myvector
3   = {1.1, 2.2, 3.3};
4 for ( auto &e : myvector )
5   e *= 2;
6 println( "{}",myvector.at(2) );
```

Output:

```
1 6.6
```

(Can also use `const auto& e` to prevent copying, but also prevent altering data.)

13. Indexing the elements

You can write an indexed for loop, which uses an index variable that ranges from the first to the last element.

```
1 for (int i= /* from first to last index */ )
2   // statement about index i
```

Example: find the maximum element in the vector, and where it occurs.

Code:

```
1 // array/vectoridxmax.cpp
2 int tmp_idx = 0;
3 int tmp_max = numbers.at(tmp_idx);
4 for (int i=0; i<numbers.size(); ++i) {
5   int v = numbers.at(i);
6   if (v>tmp_max) {
7     tmp_max = v; tmp_idx = i;
8   }
9 }
10 println( "Max: {} at index: {}",
11          tmp_max, tmp_idx );
```

Output:

```
1 Max: 6.6 at
   ↪ index: 3
```

14. Do Not Repeat Yourself

- A `vector` 'knows' its size;
- storing the size in a variable is redundant;
- $\Rightarrow$ do not repeat yourself; use the `size` method.

15. A philosophical point

Conceptually, a vector can correspond to a set of things, and the fact that they are indexed is purely incidental, or it can correspond to an ordered set, and the index is essential. If your algorithm requires you to access all elements, it is important to think about which of these cases apply, since there are two different mechanism.

Exercise 2

Normalization:

```
1 vector<float> x; // initialize!  
2 normalize(x);  
3 cout << norm(x); // should give 1
```

- Write the `norm` and `normalize` functions.

$$\|x\| = \sqrt{\sum x_i^2}$$

- Is there another possibility for `normalize` than the above?
Arguments for/against?

Exercise 3

Use general p -norm:

$$\|x\|_p = \sqrt[p]{\sum x_i^p}$$

- Read the value of p ;
- Change the syntax to

```
1 normalize( x,norm_function )
```

where *norm_function* computes the p -norm. (Hint: use a lambda expression so that you can call

```
1 norm_function( x );
```

16. Vector copy

Vectors can be copied just like other datatypes:

Code:

```
1 // array/vectorcopy.cpp
2 vector<float> v(5,0), vcopy;
3 v.at(2) = 3.5;
4 vcopy = v;
5 vcopy.at(2) *= 2;
6 println( "{}", v.at(2), vcopy.at(2)
           );
```

Output:

```
1 3.5,7
```

17. Vector methods

A vector is an object, with methods.

Given `vector<sometype> x`:

- Get elements, including bound checking, with `ar.at(3)`.
Note: zero-based indexing.
(also get elements with `ar[3]`: see later discussion.)
- Size: `ar.size()`.
- Other functions: `front`, `back`, `empty`, `push_back`.
- With iterators (see later): `insert`, `erase`

18. Your first encounter with templates

`vector` is a 'templated class': `vector<X>` is a vector-of-`X`.

Code behaves as if there is a class definition for each type:

```
1 class vector<int> {  
2 public:  
3     size(); at(); // stuff  
4 }
```

```
1 class vector<float> {  
2 public:  
3     size(); at(); // stuff  
4 }
```

Actual mechanism uses templating: the type is a parameter to the class definition.

Dynamic behaviour

19. Dynamic vector extension

Extend a vector's size with `push_back`:

Code:

```
1 // array/vectorend.cpp
2 vector<int> mydata(5,2);
3 println( "{}",mydata.size() );
4 // last element:
5 println( "{}",mydata.back() );
6 mydata.push_back(35);
7 println( "{}",mydata.size() );
8 // last element:
9 println( "{}",mydata.back() );
```

Output:

```
1 5
2 2
3 6
4 35
```

Similar functions: `pop_back`, `insert`, `erase`.

Flexibility comes with a price.

20. When to push back and when not

Known vector size:

```
1 int n = get_inputsize();
2 vector<float> data(n);
3 for ( int i=0; i<n; i++ ) {
4     auto x = get_item(i);
5     data.at(i) = x;
6 }
```

Unknown vector size:

```
1 vector<float> data;
2 float x;
3 while ( next_item(x) ) {
4     data.push_back(x);
5 }
```

If you have a guess as to size: `data.reserve(n)`.

(Issue with array-of-object: in left code, constructors are called twice.)

21. Filling in vector elements

You can push elements into a vector:

```
1 // array/arraytime.cpp
2 vector<int> flex;
3 /* ... */
4 for (int i=0; i<LENGTH; ++i)
5     flex.push_back(i);
```

If you allocate the vector statically, you can assign with `at`:

```
1 // array/arraytime.cpp
2 vector<int> stat(LENGTH);
3 /* ... */
4 for (int i=0; i<LENGTH; ++i)
5     stat.at(i) = i;
```

22. Filling in vector elements

With subscript:

```
1 // array/arraytime.cpp
2 vector<int> stat(LENGTH);
3 /* ... */
4 for (int i=0; i<LENGTH; ++i)
5     stat[i] = i;
```

You can also use new to allocate*:

```
1 // array/arraytime.cpp
2 int *stat = new int[LENGTH];
3 /* ... */
4 for (int i=0; i<LENGTH; ++i)
5     stat[i] = i;
```

*Considered bad practice. Do not use.

23. Timing the ways of filling a vector

```
1 Flexible time: 2.445
2 Static at time: 1.177
3 Static assign time: 0.334
4 Static assign time to new: 0.467
```

Vectors and functions

24. Vector as function return

You can have a vector as return type of a function.

Example: this function creates a vector, with the first element set to the size:

Code:

```
1 // array/vectorreturn.cpp
2 vector<int> make_vector(int n) {
3     vector<int> x(n);
4     x.at(0) = n;
5     return x;
6 }
7 /* ... */
8 vector<int> x1 = make_vector(10);
9 // "auto" also possible!
10 cout << "x1 size: "
11      << x1.size() << '\n';
12 cout << "zero element check: "
13      << x1.at(0) << '\n';
```

Output:

```
1 x1 size: 10
2 zero element check:
   ↪ 10
```

25. Vector as function argument

You can pass a vector to a function:

```
1 double slope( vector<double> v ) {  
2     return v.at(1)/v.at(0);  
3 };
```

Vectors, like any argument, are passed by value, so the vector is actually copied into the function.

26. Vector pass by value example

Code:

```
1 // array/vectorpassnot.cpp
2 void set0
3 ( vector<float> v,float x )
4 {
5     v.at(0) = x;
6 }
7     /* ... */
8     vector<float> v(1);
9     v.at(0) = 3.5;
10    set0(v,4.6);
11    cout << v.at(0) << '\n';
```

Output:

1 3.5

- Vector is copied
- 'Original' in the calling environment not affected
- Cost of copying?

27. Vector pass by reference

If you want to alter the vector, you have to pass by reference:

Code:

```
1 // array/vectorpassref.cpp
2 void set0
3 ( vector<float> &v, float x )
4 {
5     v.at(0) = x;
6 }
7     /* ... */
8     vector<float> v(1);
9     v.at(0) = 3.5;
10    set0(v, 4.6);
11    cout << v.at(0) << '\n';
```

Output:

1 4.6

- Parameter vector becomes alias to vector in calling environment $\Rightarrow$ argument *can* be affected.
- No copying cost

28. Vector pass by const reference

Passing a vector that does not need to be altered:

```
1 int f( const vector<int> &ivec ) { ... }
```

- Zero copying cost
- Not alterable, so: safe!
- (No need for pointers!)